CYBERSHIELD

Team Explanation and Faculty Viva Guide

How the system works, the technologies we used, how to demonstrate it, and how to answer likely questions.

For: CyberShield Project Team

Prepared from the current project implementation

Replace this line with team names before printing or sharing.

1. One-Minute Explanation

Say this first: CyberShield is a web-based phishing URL analysis platform. A user submits a website URL, and our backend safely investigates the site using live evidence such as redirects, DNS, domain age, TLS certificate details, page forms, browser behaviour, and selected threat-intelligence signals. We combine this evidence into an explainable risk score and show why the URL was classified as Safe, Low, Medium, High, or Critical.

The important difference is that CyberShield is not based only on simple URL rules such as length or suspicious words. It actually collects technical signals from the target website and its infrastructure. This makes the result more meaningful and easier to explain to a user.

What problem are we solving?

- Users often receive suspicious links through email, messages, social media, or fake login pages.
- Many people cannot judge a URL by looking at it, especially when attackers use redirects, valid certificates, and convincing page designs.
- CyberShield gives a structured risk assessment before the user enters credentials or payment data.

2. The System in One Flow

```
User enters URL in dashboard
        |
Normalize and validate URL
        |
Block localhost / private IP / unsafe target
        |
Collect HTTP, DNS, domain, TLS, browser, HTML, form and reputation evidence
        |
Extract stable evidence-v1 features
        |
Run explainable baseline classifier
        |
Display probability, risk label, contributors and technical details
```

Key message: The safety check happens before network collection. This prevents the tool from being used to access private or local systems through a submitted URL.

3. Step-by-Step: How CyberShield Works

3.1 URL input, normalization and validation

The user enters a web address in the dashboard. The backend normalizes the value and verifies that it is a complete HTTP or HTTPS URL. Unsupported schemes and malformed inputs are rejected with a clear message. This is our first reliability and security gate.

3.2 SSRF protection: blocking unsafe targets

A URL scanner is itself a network-facing tool, so it must not blindly fetch any address a user supplies. CyberShield blocks localhost, names ending in .localhost, private IP addresses, loopback addresses, link-local addresses, and other non-public destinations. It resolves hostnames and checks their addresses before allowing analysis.

Good viva answer: This control protects against Server-Side Request Forgery, or SSRF. Without it, an attacker could try to use our backend to reach internal services that are not exposed to the public internet.

3.3 HTTP and redirect evidence

The HTTP analyzer checks whether the URL is reachable and records response behaviour. Redirect chains are relevant because phishing pages may use several redirections to hide the actual destination. If a URL is not reachable, CyberShield returns Unknown rather than incorrectly declaring it safe.

3.4 DNS, domain and TLS evidence

The DNS analyzer observes domain resolution. The domain analyzer obtains registration information such as domain age when available. Very recent domains can be a useful risk signal, but they are not proof by themselves. The TLS analyzer gathers certificate metadata. A valid HTTPS certificate helps encrypt traffic, but it does not prove that a website is legitimate; phishing sites can also use HTTPS.

3.5 Controlled browser analysis

CyberShield uses Playwright with Chromium to load the page in a controlled browser session. This is important because many websites build content with JavaScript after the first HTTP response. The browser session captures the final URL, title, HTML size, redirects, cookie count, popups, downloads, permission requests, JavaScript errors, network counts, and a screenshot where possible.

3.6 HTML, JavaScript and form analysis

After browser loading, separate analyzers inspect the rendered page. The HTML analyzer identifies structural signals such as hidden iframes and meta-refresh behaviour. The JavaScript analyzer looks for suspicious script patterns such as obfuscation. The form analyzer checks whether a login or credential form sends data to a different domain, uses insecure HTTP, uses GET for password submission, or requests OTP and payment-card data.

3.7 Feature extraction and explainable decision

The feature extractor converts raw analyzer outputs into a stable evidence-v1 feature payload. The classifier starts with a base score and adds weighted impact for evidence that is present. It converts the score into a probability and maps that probability to a risk label. It also returns the highest-impact contributors, so the user sees reasons such as a cross-domain credential form, a recently registered domain, or a threat-intelligence detection.

4. Technologies We Used and Why

Technology Stack

Technology	Where we use it	Why it is useful
HTML, CSS, JavaScript	Frontend pages and dashboard	Creates a responsive, multi-page web interface.
Python	Backend services and analysis logic	Clear, modular programming for data collection and APIs.
FastAPI	REST API backend	Fast request handling, validation, and automatic API documentation.
Pydantic	Request/response models	Ensures structured and validated API data.
Playwright + Chromium	Browser analysis	Loads JavaScript-rendered pages in a controlled session.
httpx / requests	HTTP collection	Collects page and redirect information.
dnspython	DNS analysis	Resolves and inspects domain information.
python-whois	Domain age information	Helps identify recently registered domains where data exists.
cryptography / pyOpenSSL	TLS inspection	Reads certificate-related security metadata.
BeautifulSoup + lxml	HTML parsing	Extracts page structure and form signals.
SQLite	Development data storage	Stores local application data and history simply.
scikit-learn, pandas, joblib	Demo ML workflow	Supports versioned training experiments and model loading.
Docker Compose	Optional deployment	Makes local setup more repeatable.

What must we not claim?

We must not say that a valid TLS certificate means a website is safe. We must not say that the system guarantees detection of every phishing website. We must not say that the current baseline classifier is a production model trained on large real-world phishing data. The repository clearly states that the current development model uses synthetic data for a college demonstration. Our strongest answer is that CyberShield is an explainable evidence-collection pipeline with an honest baseline decision layer.

5. Architecture and Module Responsibilities

Main Modules

Module	Responsibility	Output
Frontend	Collect URL and display result, history, reports, accounts	User-friendly pages and requests
FastAPI API	Receives request and sends structured response	JSON API response
URL safety service	Rejects unsafe local/private targets	Allowed status or safe rejection
Orchestrator	Calls analyzers in the correct order and merges data	Analysis data object
Analyzers	Collect specialized evidence independently	HTTP, DNS, TLS, browser, form and other signals
Feature extractor	Converts raw evidence into evidence-v1 values	Model-ready features
Evidence classifier	Computes probability, risk and contributors	Explainable classification
Database/history	Preserves analysis records where enabled	Reviewable stored results

Why use an orchestrator?

The orchestrator is the central coordinator. It keeps the analysis process organized: normalize URL, validate it, apply the safety check, run evidence collectors, handle browser errors, extract features, classify, and return a standard response. This design is easier to test and maintain than putting every operation inside one large API route.

Why use multiple analyzers?

Different evidence sources change independently. For example, a browser failure should not necessarily stop DNS or TLS collection. Keeping analyzers separate makes the system modular, makes fault handling clearer, and allows future improvements such as adding a visual brand-matching analyzer or new threat-intelligence provider.

6. How to Demonstrate the Project

Recommended live demo order

1. Open the CyberShield home page and state the problem: users cannot easily judge suspicious links.
2. Open the dashboard and submit a normal public HTTPS URL. Explain that CyberShield first validates the URL and blocks private targets.

3. Show the result screen. Point out the risk label, phishing probability, evidence contributors, technical details, and screenshot if available.
4. Open history or reports to show that results can be reviewed later.
5. Try localhost or a private address only if the live UI/API supports the demonstration safely. Explain that rejection is an SSRF protection feature, not a phishing verdict.
6. Conclude with the limitation: the present classifier is a baseline demo; production use requires a validated model trained on real labelled data.

Presenter tip: During the demonstration, do not visit a real dangerous phishing page just to prove the system. Use safe public examples or controlled test pages. A responsible demonstration is a strength, not a weakness.

7. Suggested Team Roles During Viva

Speaking Plan

Team member focus	What to explain
Member 1: Problem and UI	Problem statement, target users, dashboard flow, result readability.
Member 2: Backend flow	FastAPI API, request lifecycle, orchestrator, JSON response.
Member 3: Security	SSRF protection, public-IP checks, controlled browser, non-submission of forms.
Member 4: Detection logic	Evidence sources, feature extraction, baseline scoring, contributors and limitations.
Any member: Future scope	Real labelled dataset, model metrics, reputation APIs, queueing and deployment hardening.

Useful handoff sentence

After explaining your part, say: “This is how the [module] works. Now [team member] will explain how its output is used in the next stage of the pipeline.” This makes the presentation sound coordinated.

8. Likely Mentor and Faculty Questions

Q: What is the main objective of CyberShield?

Answer: To help a user assess a submitted web URL using live, explainable evidence before interacting with a potentially deceptive page.

Q: Why is this better than checking URL length or keywords?

Answer: Those rules are easy to bypass and provide weak explanations. CyberShield combines infrastructure, redirect, browser, DOM, form, and optional reputation signals.

Q: What is SSRF and how do you prevent it?

Answer: SSRF is when a server is tricked into making requests to internal resources. We block localhost, private, loopback, link-local, and non-public resolved IP addresses before analysis.

Q: Why do you use Playwright?

Answer: Many modern phishing pages are JavaScript-rendered. Playwright lets us observe the rendered DOM and browser behaviour in a controlled browser context.

Q: Do you submit forms or enter credentials during analysis?

Answer: No. We observe form structure and action targets only. We do not submit target-site forms or enter user credentials.

Q: Does HTTPS mean the URL is safe?

Answer: No. HTTPS only secures communication between the browser and site. A phishing site can also have a valid certificate, so we treat TLS as one signal among many.

Q: How does the risk score work?

Answer: The evidence baseline starts from a base score and adds weighted impact for observable risk signals. It returns a probability, a label, and the most influential contributors.

Q: Is the ML model production ready?

Answer: No. The current model is explicitly a college-demo baseline trained from synthetic data. Production blocking requires lawful labelled data, validation, versioning, and measured precision/recall.

Q: What happens if a website cannot be reached?

Answer: CyberShield returns Unknown or an error state. It does not convert missing evidence into a Safe result.

Q: What are the limitations?

Answer: External data can be unavailable, websites can block automation, and a baseline model can have false positives and false negatives. The tool is decision support, not a guarantee.

Q: Why store history?

Answer: History helps users revisit analyses, supports transparency, and provides a foundation for later reporting and evaluation.

Q: What can you improve next?

Answer: Add opt-in reputation providers, collect labelled datasets, evaluate model metrics, use async queues for long scans, improve authorization, and add data retention controls.

9. Strong Closing Statement

Closing statement: CyberShield is not just a URL keyword checker. It is a modular, security-conscious analysis system that safely collects live web evidence, explains its risk decision, and clearly communicates the current model limitations. Our project demonstrates a practical foundation for an accountable phishing-detection platform.

10. Team Checklist Before Meeting Faculty

- Each member can explain the full pipeline in simple language.
- Each member knows why private IP addresses and localhost are blocked.
- Each member can explain why HTTPS is not proof of legitimacy.
- Each member knows the difference between evidence collection and classification.
- Each member states the baseline-model limitation honestly.
- The team uses only safe URLs or controlled pages during a demo.
- Team members know who will explain frontend, backend, security, and model logic.